Recursion

DSA27 — Lecture 03 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 3 · Fall 2026

Where We Are

Last week, in one line

Cost is counted as a function of n, and the **array** is the first structure we priced: $O(1)$ to index, $O(n)$ to insert or search.

Today: a way of **writing** algorithms — and a new way of **counting** what they cost.

Today

1. A function defined in terms of itself — the two rules
2. What really happens: **the call stack**
3. Recursion over numbers, over arrays, over nested data
4. The cost of recursion: **recurrences** and **recursion trees**
5. The same answer at very different costs: Fibonacci, and power
6. **Branching** recursion: Hanoi, and every choice
7. Recursion or a loop?

The Idea

A definition that uses itself

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

```
def factorial(n):
    if n == 0:                          # base case: answer directly
        return 1
    return n * factorial(n - 1)         # recursive case: a SMALLER problem
```

A **recursive** function solves a problem by solving a **smaller instance of the same problem**, and building its answer from that.

The two rules

Every recursive function needs:

1. **A base case** — an input small enough to answer **without** recursing.
2. **A recursive case that makes progress** — every call must move **closer to the base case**.

Break rule 1, or rule 2, and the calls never stop:

```
>>> factorial(-1)          # -1, -2, -3, ... never reaches 0
RecursionError: maximum recursion depth exceeded
```

Trust the recursion

To write the recursive case, **assume the recursive call already works** — for the smaller input — and ask only:

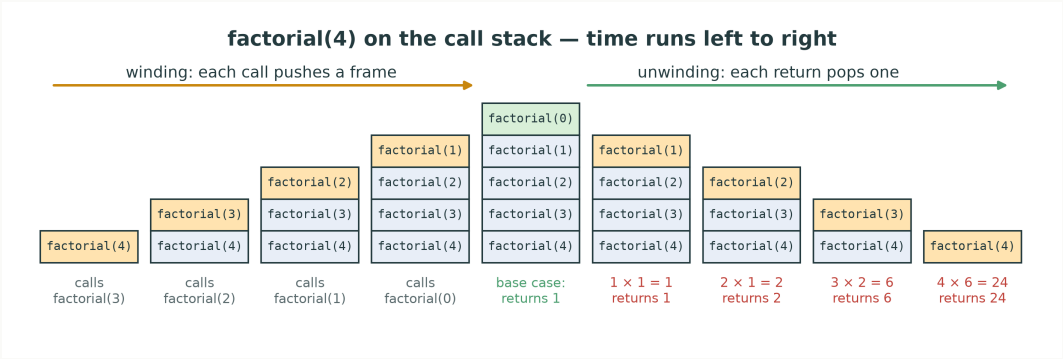
Given the answer for the smaller problem, how do I build the answer for mine?

For `factorial(n)`: "if I had `factorial(n - 1)`, I would multiply it by n."

Do **not** trace every call in your head. That is the computer's job.

The Call Stack

Every call gets a frame



Each call pushes a **frame** holding its own n and where to return. The base case stops the growth; then each `return` pops a frame and hands its value down.

Watch it run: the Recursion Lab

Recursion Lab — open it in any browser.

- The code, the **call stack** and the **recursion tree**, one step at a time
- **Step over** a call: trust the recursion
- *What happens next?* — answer before you step
- Nine scenarios that **go wrong**: no base case, no progress, a missing `return`, a wrong base value, hidden $O(n^2)$…

Depth is space

The call stack is memory. A recursion that goes **d** calls deep holds **d** frames at once:

Function	Deepest point	Extra space
`factorial(n)`	n + 1 frames	$O(n)$
`power(x, n)` by halving	about $\log_2 n$ frames	$O(\log n)$
`hanoi(n)`	n frames	$O(n)$ — though it makes 2^n calls

```
>>> import sys
>>> sys.getrecursionlimit()
1000
```

Recursion in Practice

Over numbers: shrink the number

```
def count_digits(n):
    """How many decimal digits in a non-negative int."""
    if n < 10:                          # one digit
        return 1
    return 1 + count_digits(n // 10) # drop the last digit
```

```
def to_binary(n):
    """'101' for 5. n >= 0."""
    if n < 2:
        return str(n)
    return to_binary(n // 2) + str(n % 2)
```

The measure: **n itself**, divided by 10 or 2 each call — so these are $O(\log n)$ deep.

Over an array: shrink the range, not the array

```python
def array_max(arr, i=0):
    """Largest value in a non-empty Array, from index i on."""
    if i == len(arr) - 1:                    # one element left
        return arr[i]
    rest = array_max(arr, i + 1)             # the answer for the rest
    return arr[i] if arr[i] > rest else rest
```

The recursion moves an **index**; the array never changes. $O(n)$ time, $O(n)$ stack.

Over nested data: the data is recursive

```
def depth(nested):
    """How deeply lists are nested. depth([1, [2, [3]]]) == 3."""
    if not isinstance(nested, list):
        return 0                              # a plain value: no list at all
    deepest = 0
    for item in nested:
        d = depth(item)
        if d > deepest:
            deepest = d
    return 1 + deepest
```

A list of lists of lists… is defined in terms of itself — so the function is too.

What Recursion Costs

Recurrences

Write the cost **T(n)** in terms of the cost of the smaller calls:

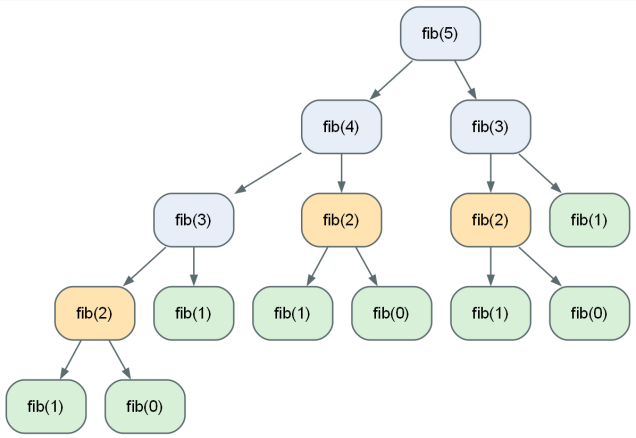
Code shape	Recurrence	Solution
one call on n – 1, O(1) work	$T(n) = T(n-1) + c$	$O(n)$
one call on n / 2, O(1) work	$T(n) = T(n/2) + c$	$O(\log n)$
one call on n – 1, **O(n) work** (a slice)	$T(n) = T(n-1) + cn$	$O(n^2)$
two calls on n – 1	$T(n) = 2T(n-1) + c$	$O(2^n)$
two calls on n – 1 and n – 2	$T(n) = T(n-1) + T(n-2) + c$	$O(1.62^n)$
two calls on n / 2, O(n) work	$T(n) = 2T(n/2) + cn$	$O(n \log n)$

Solving one by unrolling

$$
\begin{aligned}
T(n) &= T(n-1) + c \\
&= T(n-2) + 2c \\
&= T(n-3) + 3c \\
&\vdots \\
&= T(0) + nc \;=\; O(n)
\end{aligned}
$$

$$
\begin{aligned}
T(n) &= T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \ldots \\
&= T(1) + c \log_2 n \;=\; O(\log n)
\end{aligned}
$$

Solving one with a tree



fib(5): 15 calls. fib(2) alone is computed 3 times (highlighted). Leaves are base cases.

`fib(n) = fib(n-1) + fib(n-2)` — **two** calls, on inputs almost as big.

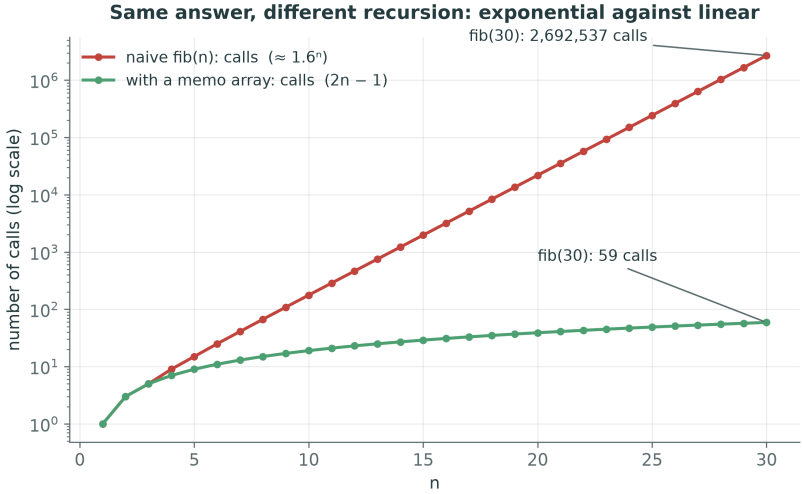
Same answer, one array: memoisation

```
from dsa.array import Array

def fib_memo(n, memo=None):
    if memo is None:
        memo = Array(n + 1)                # memo[k] will hold fib(k)
    if n < 2:
        return n
    if memo[n] is None:                    # not computed yet
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]
```

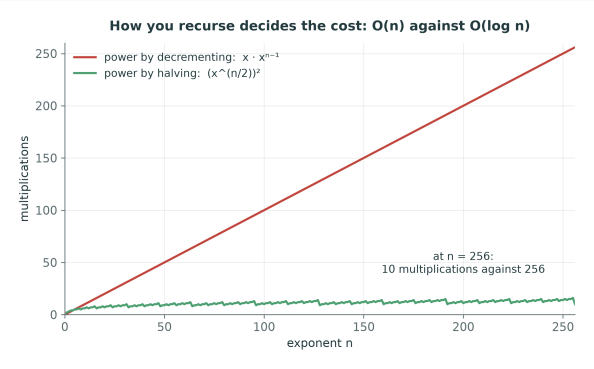
Compute each `fib(k)` **once**, store it, look it up after that.

Exponential against linear



How you recurse decides the cost

$$x^n = x \cdot x^{n-1} \qquad \text{versus} \qquad x^n = \left(x^{n/2}\right)^2 \quad (\times x \text{ if n is odd})$$



$T(n) = T(n-1) + c = O(n)$ against $T(n) = T(n/2) + c = O(\log n)$.

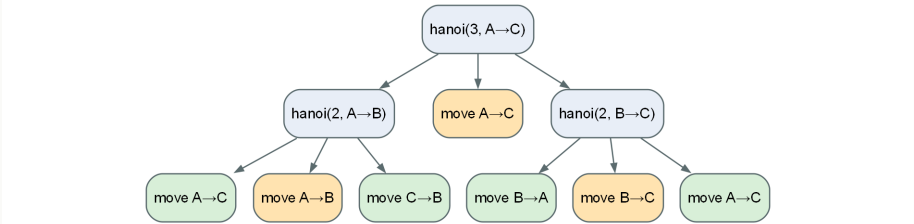
Branching Recursion

Towers of Hanoi

Move n disks from peg A to peg C, one at a time, never a larger disk on a smaller one.

1. Move the top **n – 1** disks from A to **B** (out of the way) — *recursively*
2. Move the **largest** disk from A to C — *one move*
3. Move the **n – 1** disks from B to **C**, on top of it — *recursively*

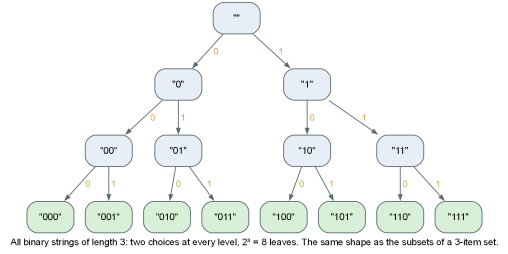
$$T(n) = 2T(n-1) + 1 \quad \Rightarrow \quad T(n) = 2^n - 1 \text{ moves}$$



hanoi(3): move 2 out of the way, move the largest, move 2 back on top — 7 moves = $2^3 - 1$. Read the moves left to right along the bottom.

Every choice: two branches per decision

```
def binary_strings(n):
    """Every string of n bits, e.g. n=2 -> ['00', '01', '10', '11']."""
    if n == 0:
        return [""]                              # one way to choose nothing
    shorter = binary_strings(n - 1)
    return ["0" + s for s in shorter] + ["1" + s for s in shorter]
```



All binary strings of length 3: two choices at every level, $2^3 = 8$ leaves. The same shape as the subsets of a 3-item set.

n decisions, 2 choices each: 2^n results, and at least $O(2^n)$ time just to write them down.

Merging: recursion on two inputs

To merge two sorted lists:

- If either is empty, the answer is the other one.
- Otherwise, the **smaller head** comes first, followed by the merge of everything else.

Measure: `len(left) + len(right)`, one less each call — $O(n + m)$ calls.

Recursion or a Loop?

Anything recursive can be a loop

- **Tail-shaped** recursion (one call, nothing done after it) is a loop in disguise:

```
def gcd_loop(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

- **Branching** recursion becomes a loop **plus your own stack** — which is what the call stack was doing for you. (Week 14: `dfs` and `dfs_iterative`.)

When to choose which

Recursion, when…	A loop, when…
the data is recursive: trees, nested lists, grammars	the data is flat: arrays, ranges
the problem splits in halves: binary search, merge sort	the recursion would be one call deep per element
it branches: all subsets, Hanoi, backtracking	depth could exceed ~1,000
clarity matters more than the last 20% of speed	you are in a hot loop in Python

Recursion is a way of thinking first, and a way of writing code second.

This Week

Exercises: `dsa/recursion.py`

Function	Idea	Target
`factorial(n)`	n · (n − 1)!; reject n < 0	$O(n)$
`sum_digits(n)`	last digit + the rest	$O(\log n)$
`gcd(a, b)`	Euclid: gcd(b, a mod b)	$O(\log \min(a, b))$
`power(x, n)`	halve the exponent — **once**	$O(\log n)$
`reverse_string(s)`	last + reverse(rest)	—
`is_palindrome(s)`	ends match, recurse on the middle	—
`flatten(nested)`	recurse into lists only	—
`merge_sorted(l, r)`	smaller head first; stable	$O(n + m)$
`hanoi(n, …)`	n − 1 away, largest, n − 1 back	$O(2^n)$
`subsets(items)`	out, or in	$O(2^n)$ results
`permutations(items)`	which goes first?	$O(n!)$ results

```
pytest tests/test_recursion.py -v
```

Homework 3 — before Lecture 04

1. **Implement** `dsa/recursion.py` until `pytest tests/test_recursion.py` passes.
2. **Trace** `hanoi(3, "A", "C", "B")` by hand: draw the call tree and list the moves in order. Check against your code.
3. **Recurrences.** For each of `sum_digits`, `power`, `hanoi` and `permutations`, write the recurrence and solve it.
4. **Measure** `fib` (naive) and `fib_memo` from the handout with `viz.complexity.measure` for n = 5, 10, 15, 20, 25, and plot both. At what n does the naive one take longer than a second on your machine?
5. **Lab 03** exercises, if not finished in the lab.

Summary

Seven things to keep

1. Recursion = a **base case** + a recursive case that **makes progress**.
2. **Trust the recursion**: assume the smaller call works; build on it. That is induction.
3. Each call gets a **frame** on the **call stack**; stack space = the **deepest** point. Python stops at ~1,000.
4. The cost is a **recurrence**: how many calls, on what size, how much work outside them.
5. $T(n-1)+c \to O(n) \cdot T(n/2)+c \to O(\log n) \cdot 2T(n-1)+c \to O(2^n)$.
6. **How** you recurse decides the cost: fast power, memoised Fibonacci. Slicing hides $O(n)$.
7. **Branching** recursion explores every choice — 2^n subsets, $n!$ orderings.

Next

Week 4 — Dynamic arrays. An array that grows: a fixed `Array`, replaced by a bigger one when it fills — and why doubling makes append **amortised** $O(1)$.